Coursework 1: GitHub Portfolio

Course: COMP08101 Programming for Cyber Security

Topic: Coursework 1

Duration: Home Assignment

Instructor: Raman Singh/ Alan Hart

Submission Deadline: 13th November 2026 (11:55 PM)

Overall Assessment: 40%

Marks Assigned for this part: 100

Coursework Overview:

This coursework assesses your understanding of the Python programming concepts and software development practices introduced during Weeks 1–6 of the module Programming for Cyber Security. You will complete the programming exercises given in the "list of exercises" section. Each selected exercise must be implemented in Python, documented appropriately, and maintained in a dedicated Git branch within your GitHub repository.

You must complete this coursework independently. All submitted programs must represent your own work and demonstrate your understanding of the programming concepts taught during the module. A form given on Brightspace under "Coursework1" must be filled out to submit the GitHub link and your basic details. Without this form, it will not be possible to identify your portfolio and hence assess your work.

List of Exercises:

The portfolio should contain the following exercises:

Task 1: Security Tool

You are developing a basic security tool for a small organisation. When the program starts, the user must authenticate. After successful authentication, the user is presented with a security menu. The user can then select different security functions. The program should continue running until the user chooses Exit.

The tool will have the following functionalities:

1. Security Event Counter
2. Brute-Force Simulation
3. Login Authentication
4. Exit

The program should perform the following functions:

1. **Security Event Counter:** The program should ask for a password 10 times. There will be a correct password stored in a variable. You need to count the number of times correct and incorrect passwords are entered. Also display the percentage of incorrect passwords entered and raise an alert if this is more than 50%.

2. **Brute-Force Simulation:** Use a predefined password. Allow a maximum number of attempts. Ask the user for a password guess. Count attempts. Stop when the correct password is entered. Stop when the maximum number of attempts is reached.
3. **Login Authentication:** Ask for a username and password. Check the credentials. Display whether authentication succeeded or failed. If it fails, ask for the username and password again. Allow three failed logins. If failed logins reach the maximum attempts, display a message that the account is frozen for 24 hours.
4. **Exit:** If a user selects exit, display a message that says, " Thank you for using the security tool".

Task 2: Security Event Logger

A security event logger is a program or system that records security-related activities so they can be reviewed later. Security event logs can help organisations to monitor login activity, detect suspicious or repeated failed logins, investigate security incidents, keep a record of system activity and determine when an event occurred.

For example, if someone tries to log in with the wrong password, the logger might record:

Event ID: 3971

Username: admin

IP Address: 192.168.1.25

Event: Failed login

Timestamp: 2026/-09-27 01:35:42

Write a program that will ask for a username and password from a user until the user enters the correct username and password. Print the event log for each try, including all failed and successful login attempts.

Task 3: IP Address Validator (using Function)

Create a basic IP address validation function using string operations. IP addresses are commonly found in security logs and network monitoring data. Before

analysing an IP address, a program can perform basic checks to determine whether the input has the expected structure.

In this task, you will create a basic format check. You are not expected to implement a complete IPv4 validation system. Create a function called *validate_ip()*.

The function should:

- Accept an IP address as a parameter.
- Remove unnecessary spaces.
- Check whether the IP address contains periods/dots (.).
- Check whether it contains exactly three periods/dots.
- Check if we have 4 numbers in the IP address.
- Check if all numbers extracted between periods/dots are less than 256 and more than or equal to 0 (value should be between 0 and 255).
- Return True if these basic requirements are met. Return False otherwise.

The user will enter an IP address and will get an appropriate message indicating whether the IP address is valid or not.

- **Example Output 1**

Enter an IP address: 192.168.1.10

Valid IP address

- **Example Output 2**

Enter an IP address: 192.168.1.300

Invalid IP address

- **Example Output 3**

Enter an IP address: 192.168.1

Invalid IP address

- **Example Output 4**

Enter an IP address: 192.168.one.10

Invalid IP address

Task 4: Security Incident Manager using Dictionaries

You are creating a program that stores and updates information about a security incident.

1. Create a dictionary containing a username, IP address, event type and risk level.
2. Display the username and IP address.
3. Add a new field called status.
4. Update the risk level.
5. Remove one field from the dictionary.
6. Display all remaining keys and values.
7. Display the total number of key-value pairs.
8. Add appropriate comments in the code for better readability.

Task 5: Reusable Cyber Security Utilities using User-Defined Modules

You are developing a small collection of reusable Cyber Security functions. Instead of placing all the code in one file, you will organise it into a separate Python module.

1. Create a Python file called *security_tools.py*.
2. Define a function that validates a password for minimum length and for variety (at least one character, digit and symbol).
3. Define another function that performs a basic check of an IP address, such as checking its format (three dots, 4 numbers/octets and numbers in the range between 0 and 255).
4. Define a third function that can generate a strong random password. A strong password is a long, unpredictable, and unique combination of characters. Your function should generate a strong password of length between 10 and 16 characters (use a random function to decide the length of the password every time), and should have variety (a mix of capital letters, lowercase letters, digits, and symbols).
5. Create a second file called *main.py*.
6. Import *security_tools* into *main.py*.
7. Ask the user to enter a password and an IP address. Check if the password and IP address are valid. If the password provided by the user is not strong, suggest a randomly generated strong password.
8. Call the functions from the module and display the results.
9. Add appropriate comments in the code for better readability.

Important Instructions and GitHub Repository Requirements

1. You must create a GitHub repository with the exact name: "prog-for-cyber-cw1"
2. The repository must be created under your own GitHub account.
3. The repository visibility should be public.
4. Main branch: The main branch must act as the central overview of your coursework portfolio. The main branch should provide a clear starting point that contains an overview of the coursework. It must contain:
   - A README.md file describing the coursework repository.
   - Your student identification details.
   - A brief description of every selected exercise.
   - A list of the corresponding Git branches.
5. Branch requirements: Create a separate Git branch for each selected programming exercise, using the branch names specified below.
   - Task 1 branch name: **security-tool**
   - Task 2 branch name: **security-event-logger**
   - Task 3 branch name: **ip-address-validator**
   - Task 4 branch name: **security-incident-manager**
   - Task 5 branch name: **cyber-security-utilities**
6. Each branch must contain the Python implementation and its own README.md file. Please add your name, your campus and your banner ID in the README of all branches. Explain what the program does and provide instructions to run the program.
7. All Python programs must be readable, logically organised, and appropriately commented.
8. Use comments to explain the purpose of important sections of code, the logic behind conditional statements and loops, the purpose of functions,

their parameters and return values, any assumptions or validation performed by the program, etc.

9. Maintaining code readability is important; hence, your code must:
    - Use meaningful variable and function names.
    - Follow consistent indentation.
    - Avoid unnecessary repetition.
    - Use an appropriate structure for the task.
    - Handle expected invalid input or errors where relevant to the exercise. Please use try-except in all tasks wherever it is required.
    - Avoid including unused code or unused variables.
10. Include an appropriate LICENSE file in all branches.
11. The use of AI-generated code is strictly prohibited in this coursework 1.
12. Testing and Providing Evidence: Test your tasks by running them in VS Code. Test for both normal/expected input as well as invalid input. Take a snapshot/screenshot of the output and push that snapshot/screenshot to the GitHub branch of that task to make it a part of your branch.
13. Finally, once your GitHub repo is ready (along with all branches), submit the "Coursework 1 submission form" given in the Brightspace Coursework 1 link. Provide your name, campus, Banner ID and full GitHub profile link. If you fail to submit this form, your Coursework 1 will not be marked.
14. Do not modify/add/delete anything in the GitHub repo or any branches after the deadline. Any modification after the deadline will be considered a late submission.

Marking Rubric:

Sr. No.	Criteria	Marks	Description
1	Python program functionality	60	Correctness, completeness and appropriate use of Python concepts in the selected exercises; successful execution evidence provided; program tested appropriately for valid and invalid input; output screenshots provided.
2	Code quality and comments	15	Readability, meaningful naming, appropriate structure and comments explaining important instructions and functions; output screenshots provided
3	README documentation and License	15	Clear program descriptions, execution instructions, relevant concepts, examples where appropriate, and student identification details in every required README, appropriate licenses added in each branch
4	Git and GitHub management	10	Correct repository name, required branches, appropriate commits, successful pushes, and clear organisation of the portfolio
	Total	100	